

Task 4

Observability Lab

Overview

This task focuses on implementing of Prometheus and Grafana to monitor the services created in the previous tasks by making a few modifications. Prometheus metrics are instrumented in REST and gRPC services. Prometheus is used to scrape metrics from these services, and Grafana is used to visualize “Golden Signals” (latency, traffic, and errors) by creating a live dashboard. OpenTelemetry signals are scraped and forwarded to Jaeger for tracing.

Github Repository: <https://github.com/bavanmendon/dsa-practicals/tree/master>

REST Service Modifications

‘prometheus_client’ is added to REST requirements file as it will be used to scrape metrics from the REST service. The service is updated with Prometheus metrics and hooks. The metrics include latency and counter. When the service encounters a new incoming request, the latency timer begins, and after the request has been processed, the timer is reset, and the counter is incremented by the number of requests processed, and eventually the response is sent back to the user. The difference between start and stop time for each request gives the latency. A new endpoint ‘/metrics’ is added, where all registered Prometheus metrics are collected and displayed in text format. This service is hosted on port 5000. OpenTelemetry Tracing has been added for tracing requests using Jaeger.

gRPC Service Modifications

‘prometheus_client’ and ‘py-grpc-prometheus’ libraries are added to the gRPC requirements file. In the gRPC server, a Prometheus server interceptor is added, and the metrics are exposed via port 9103, where the Prometheus HTTP server is started. This port connects the gRPC server to Prometheus and lists all Prometheus metrics relating to the gRPC server. The gRPC server is hosted on port 50051 for communication with the REST service.

Prometheus Configuration

A new directory ‘/observability’ is created within the project where the configuration file ‘prometheus.yml’ is created. This is used to decide on details such as scrape intervals and evaluation intervals. In this context, the metrics from the REST service are directly scraped from port 5000, and gRPC metrics are scraped through port 9103.

Datasource Configuration (Grafana)

New directories ‘/observability/grafana-provisioning/datasources’ is created within ‘/observability’, where the datasource configuration ‘datasource.yml’ is created. This file automatically configures the Prometheus datasource without a requirement for manual UI. It also contains the port on which the Prometheus server (Query API and Web UI) is hosted, and it is done on port 9090.

Dashboard Configuration (Grafana)

New directory '/dashboards' is created within '/grafana-provisioning', where Golden Signals (traffic, errors, latency) are automatically loaded from 'golden-signals.json', which is used to create a visual layout with graphs in four panels on port 3000 with the following data:

- **Panel 1:** Shows the traffic from the REST service.
`sum(rate(http_requests_total[1m])) by (endpoint)`
- **Panel 2:** Shows the histogram of latency per request.
`histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, endpoint))`
- **Panel 3:** Shows the error rate.
`(sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))) * 100`
- **Panel 4:** Shows the traffic from the gRPC service.
`histogram_quantile(0.95, sum(rate(grpc_server_handling_seconds_bucket[5m])) by (le, grpc_method))`

OpenTelemetry Tracing Configuration and Jaeger

'otel-collector-config.yml' is created to listen for OTLP traces on port 4317, and forwards it Jaeger. Jaeger provides a UI to view the traces, which is hosted on port 16686.

Docker Compose Modifications

Services for Prometheus, Grafana, OpenTelemetry, and Jaeger are added here on ports 9090, 9130, 4317 and 16686, respectively. Also, the gRPC service is exposed to port 9103 for scraping by the Prometheus HTTP server. The images for Prometheus, Grafana, OpenTelemetry, and Jaeger are downloaded from Docker's Hub remotely.

Load Script

A script is written to send 100 '/POST' and '/GET' requests to the REST service with 1 second interval between each '/POST'.

Control Flow

- The docker compose file is used to build and run all five services: rest-service, grpc-service, mongodb, prometheus, and grafana.
- User sends the request load to the REST service on port 5000.
- The timer begins, and the data is sent to the gRPC service on port 50051 and then stored using the MongoDB service on port 27017, which concludes the processing of the request. The latency is calculated after processing, and a response is sent to the user.
- Prometheus is hosted on port 9090 and scrapes REST service metrics directly from port 5000 and gRPC service metrics from port 9103, where the HTTP server is started.
- Grafana is hosted on port 3000 and gets the data from Prometheus on port 9090, as it has a native plugin that enables it to communicate without a custom code requirement.

- The graphs can be seen with live-time changes across four panels under the ‘Dashboard’ section after logging into Grafana via ‘<http://localhost:3000>’.
- REST service sends OTLP to otel-collector on port 4317, which forwards the data to Jaeger, which is hosted on port 43177. Its UI is hosted on port 16686. The traces can be verified on Jaeger’s UI.

Test Operations and Results

Build, start, and check running containers using a compose file

Containers are created and started using the following commands:

`docker-compose build` # Builds all services

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/observe-lab$ docker-compose build
mongodb uses an image, skipping
prometheus uses an image, skipping
grafana uses an image, skipping
Building grpc-service
[+] Building 1.4s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> == transferring dockerfile: 330B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.4s
=> [internal] load .dockerignore                                  0.0s
=> == transferring context: 2B                                     0.0s
=> [internal] load build context                                  0.0s
=> == transferring context: 424B                                   0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f75559 0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY . .                                           0.0s
=> exporting to image                                              0.0s
=> == exporting layers                                             0.0s
=> == writing image sha256:6a96d3ba714836074d78a4551ee7334ac27ac44035a8f1541a4237cf63019723 0.0s
=> == naming to docker.io/library/observe-lab_grpc-service         0.0s
Building rest-service
[+] Building 0.3s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> == transferring dockerfile: 200B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.2s
=> [internal] load .dockerignore                                  0.0s
=> == transferring context: 2B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f75559 0.0s
=> [internal] load build context                                  0.0s
=> == transferring context: 320B                                   0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY . .                                           0.0s
=> exporting to image                                              0.0s
=> == exporting layers                                             0.0s
=> == writing image sha256:8f0dfa7d7cecc0d069634600f441fcde01d4600a45d8274edcea1cc9c6ff82b0 0.0s
=> == naming to docker.io/library/observe-lab_rest-service         0.0s
```

`docker-compose up -d` # Starts all services

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/observe-lab$ docker-compose up -d
Creating network "observe-lab_app-network" with driver "bridge"
Creating volume "observe-lab_mongo-data" with default driver
Creating mongodb ... done
Creating prometheus ... done
Creating grpc-service ... done
Creating grafana ... done
Creating rest-service ... done
```

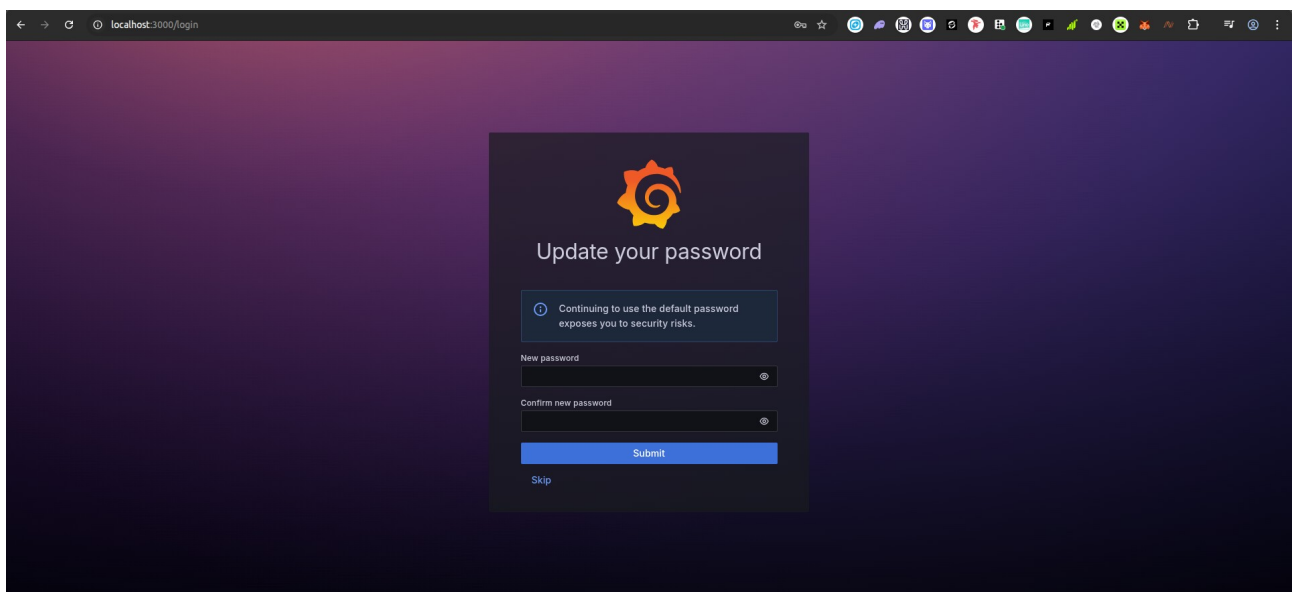
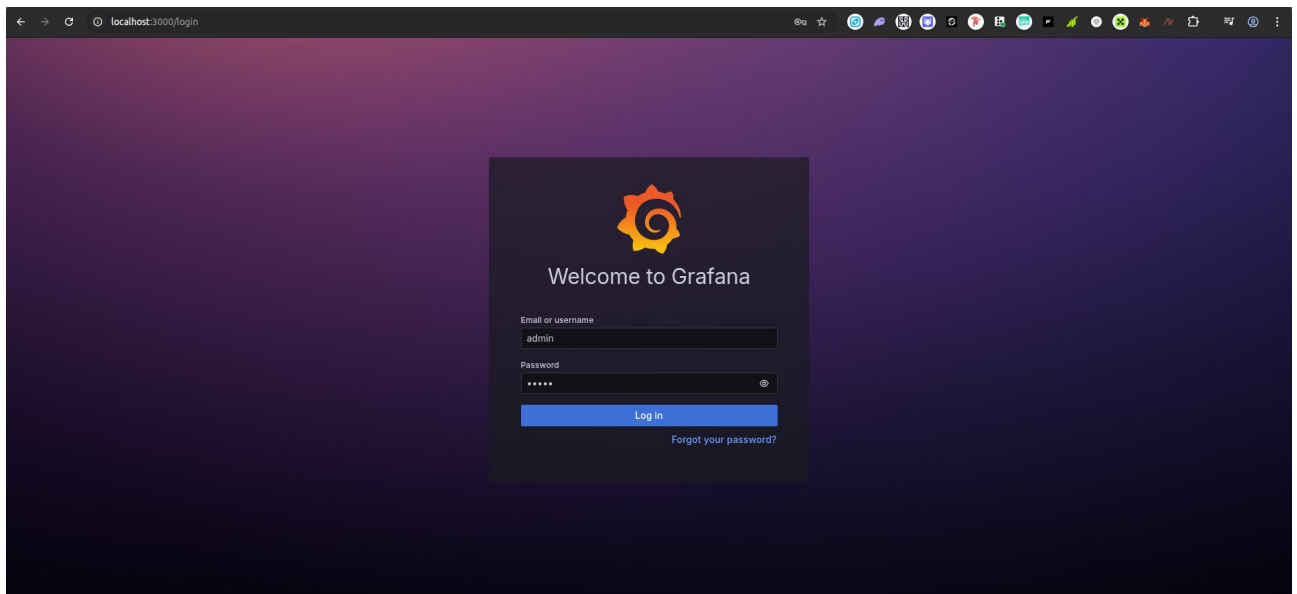
docker-compose ps

Lists all services and their states

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/obs-  
erve-lab$ docker-compose ps  
      Name                       Command                                State      Ports  
-----  
grafana        /run.sh                                Up         0.0.0.0:3000->3000/tcp, :::3000->3000/tcp  
grpc-service   python3 -u server.py                  Up         0.0.0.0:50051->50051/tcp, :::50051->50051/tcp,  
0.0.0.0:9103->9103/tcp, :::9103->9103/tcp  
mongodb        docker-entrypoint.sh mongod           Up         0.0.0.0:27017->27017/tcp, :::27017->27017/tcp  
prometheus     /bin/prometheus --config.f ...        Up         0.0.0.0:9090->9090/tcp, :::9090->9090/tcp  
rest-service   python3 app.py                        Up         0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
```

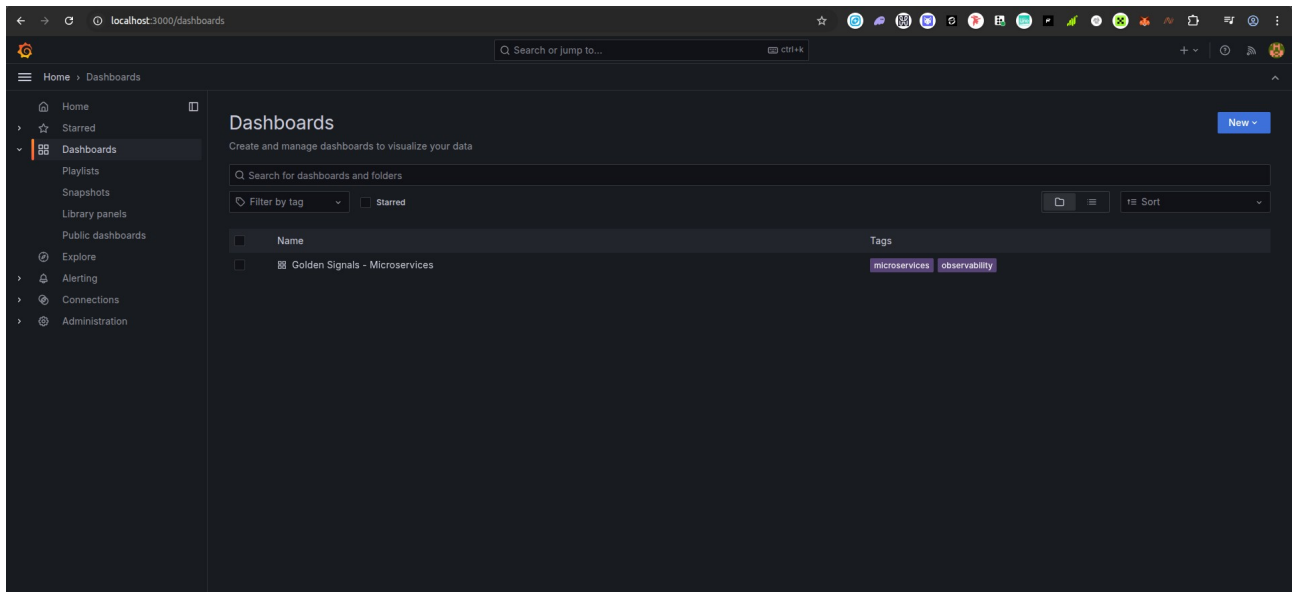
Grafana Login

On the first run, the username and password are both 'admin', and the host will offer a password change option after login.



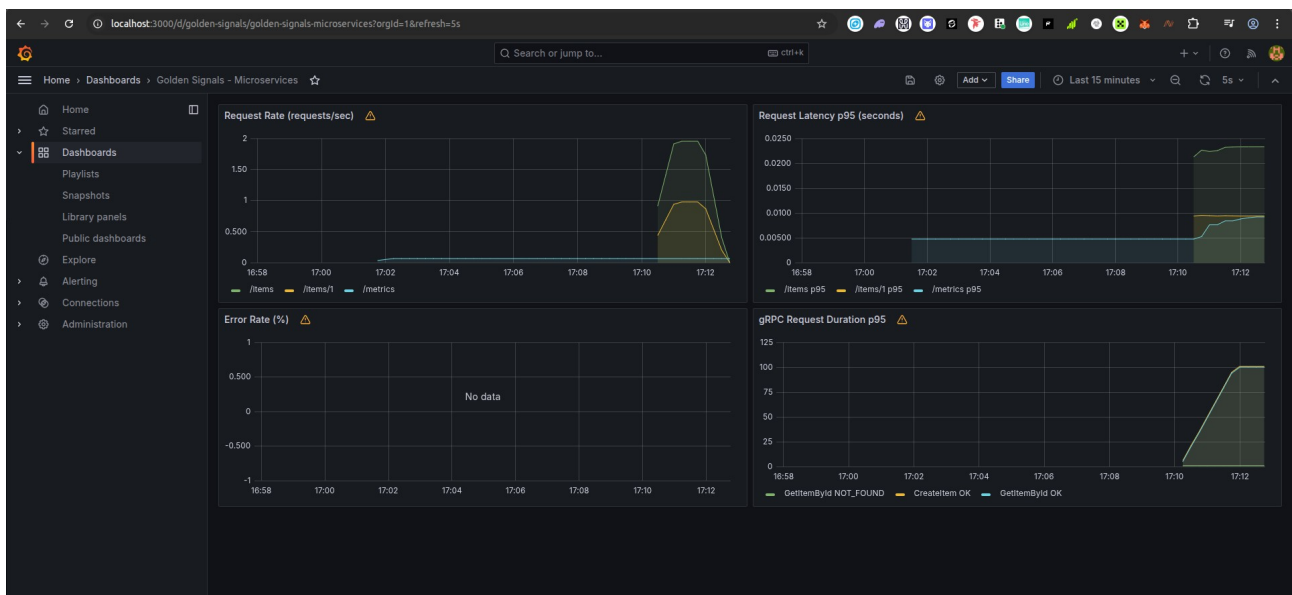
Dashboard Manager

The dashboard created with the name ‘Golden Signals – Microservices’ is shown in this section. Click on it to see all panels.



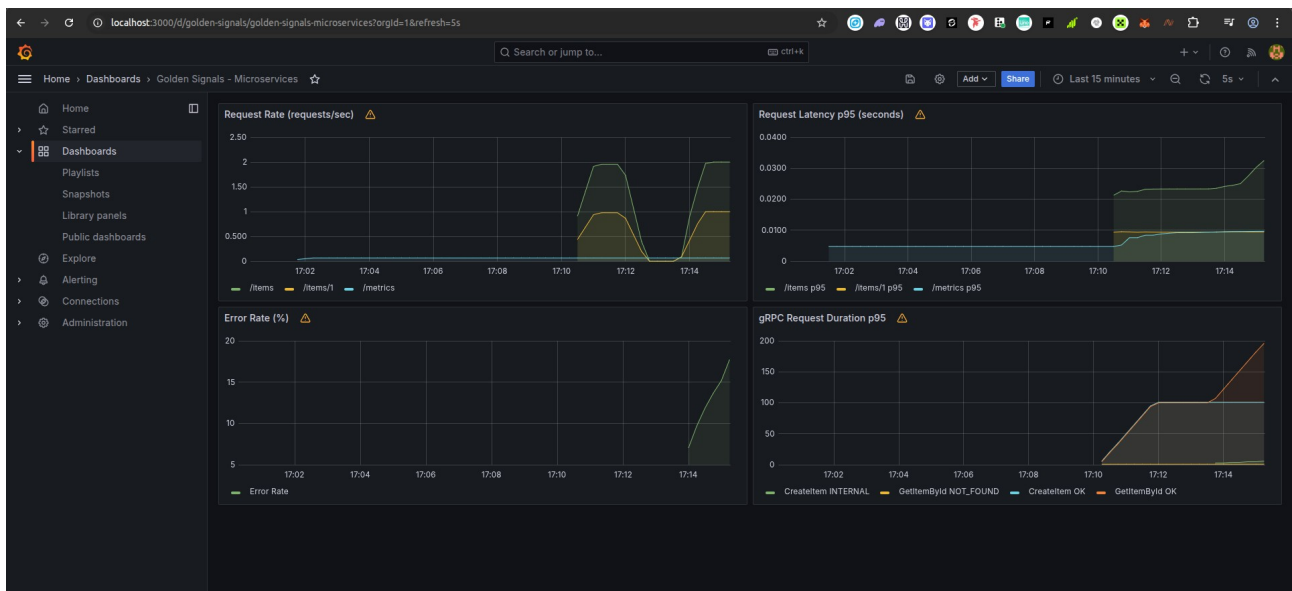
Dashboard output after Load Test

A load script is run with 100 sets of requests, which takes about 100 seconds to complete.



Dashboard Output with Intentional Errors

The same load script is run again which gives an increasing error rate since ‘/POST’ is not idempotent.



Jaeger Tracing

Tracing has been added to the REST service, and it can be selected from the drop-down menu 'Service'. The list of traces shows up on the UI.

